Componentes React, Exportações, Importações e Modularização

Ana Júlia Lima

Desenvolvido para
Young 3

Ctrl Play
Brasil
2026

Sumário

1 Introdução **3**

2 O que são componentes no React? **3**
2.1 Definição . . . 3
2.2 Ideia principal . . . 3
2.3 Exemplo conceitual . . . 3
2.4 Resumo . . . 4

3 Por que usar componentes? **4**
3.1 Reutilização . . . 4
3.2 Manutenção . . . 4
3.3 Organização . . . 4
3.4 Leitura mais fácil . . . 4
3.5 Resumo em tabela . . . 5

4 Componente Funcional **5**
4.1 O que é . . . 5
4.2 Exemplo básico . . . 5
4.3 Por que ele é chamado de funcional? . . . 5
4.4 Exemplo com mais estrutura . . . 5
4.5 Características . . . 5

5 Exportação de um Componente Funcional **6**
5.1 Declaração de função e exportação no final . . . 6
5.2 Exportação direta na declaração . . . 6
5.3 Declaração de constante e exportação no final . . . 6
5.4 Por que na arrow function com constante não exportamos default diretamente? . . . 7
5.5 Exportação direta nomeada . . . 7

6 Componente de Classe **8**
6.1 O que é . . . 8
6.2 Exemplo básico . . . 8
6.3 Como ler esse código . . . 8
6.4 Observação importante . . . 9

7 Exportação de um Componente de Classe **9**
7.1 Declaração de classe e exportação no final . . . 9
7.2 Exportação direta na declaração . . . 9
7.3 Exportação direta nomeada . . . 10

8 Importação default x importação nomeada **10**
8.1 Importação default . . . 10
8.2 Importação nomeada . . . 11
8.3 Comparação direta . . . 11
8.4 Exemplo com múltiplas exportações nomeadas . . . 11

9 Esquema mental: default x nomeada **12**

10 Diferença entre componente funcional e componente de classe **12**
10.1 Resumo . . . 12

11 Componentes reutilizáveis e claros **12**
11.1 Fazer uma única coisa bem feita . . . 12
11.2 Por que isso ajuda? . . . 13
11.3 Problema de componentes grandes demais . . . 13

12 Nomenclatura consistente **13**
12.1 O que é . . . 13
12.2 Exemplos bons . . . 13
12.3 Exemplos ruins . . . 13
12.4 Por que isso importa? . . . 14
12.5 Padrão importante em React . . . 14

13 Documentação adequada **14**
13.1 Por que documentar? . . . 14
13.2 Exemplo de comentário simples . . . 14
13.3 Documentação clara e concisa . . . 14
13.4 Exemplo ruim de documentação . . . 14
13.5 Exemplo melhor . . . 15

14 Modularização bem feita **15**
14.1 O que é modularização? . . . 15
14.2 Exemplo ruim: tudo em um arquivo . . . 15
14.3 Exemplo melhor: código modular . . . 15
14.4 Como isso arruma o código? . . . 16
14.5 Exemplo prático . . . 16
14.6 Resultado . . . 16

15 Boas práticas com componentes **16**

16 Erros comuns: **17**

17 Esquema geral do conteúdo **17**

18 Conclusão **17**

19 Exercícios **18**
19.1 Conceitos . . . 18
19.2 Exportação e importação . . . 18
19.3 Leitura de código . . . 18
19.4 Boas práticas . . . 18

20 Gabarito — Exercícios **19**
20.1 Conceitos . . . 19
20.2 Exportação e importação . . . 19
20.3 Leitura de código . . . 20
20.4 Boas práticas . . . 20

1 Introdução

Em React, um dos conceitos mais importantes é o de **componente**. Quase tudo que construímos em uma aplicação React é organizado em componentes: botões, menus, cabeçalhos, cartões de produto, formulários, barras de navegação, páginas inteiras e muito mais.

Entender componentes é essencial porque esse conceito está no centro da forma como React organiza o código. Ao aprender componentes, deixamos de enxergar a aplicação como um bloco gigante e passamos a enxergá-la como um conjunto de peças menores, cada uma com uma função específica.

O objetivo não é apenas mostrar sintaxe, mas sim entender **por que esse modelo existe e como ele melhora projetos reais**.

2 O que são componentes no React?

2.1 Definição

Um componente em React é um **bloco de código reutilizável** que representa uma parte da interface.

Podemos pensar em componentes como peças de montagem. Em vez de construir toda a aplicação em um único arquivo enorme, separamos a interface em partes menores.

Por exemplo, em uma loja virtual poderíamos ter:

- um componente para o cabeçalho;
- um componente para o menu;
- um componente para o card de produto;
- um componente para o botão de compra;
- um componente para o rodapé.

2.2 Ideia principal

Cada componente:

- tem uma responsabilidade;
- pode ser reutilizado;
- ajuda a organizar o projeto;
- facilita a manutenção;
- torna o código mais legível.

2.3 Exemplo conceitual

Imagine uma página com vários botões iguais. Em vez de escrever o mesmo código várias vezes, podemos criar um componente:

```
function Botao() {
  return <button>Clique aqui</button>;
}
```

Depois, esse componente pode ser usado em vários lugares.

2.4 Resumo

Componente é:

- uma parte da interface;
- um pedaço reutilizável do código;
- uma forma de dividir o projeto em partes menores.

3 Por que usar componentes?

3.1 Reutilização

Quando um mesmo elemento aparece várias vezes, não faz sentido copiar e colar o mesmo código muitas vezes.

Se criarmos um componente, podemos reutilizá-lo:

```
function CardProduto() {
  return (
    <div>
      <h2>Produto</h2>
      <button>Comprar</button>
    </div>
  );
}
```

Esse componente pode aparecer em várias partes da aplicação.

3.2 Manutenção

Quando o código está repetido em muitos lugares, qualquer alteração dá mais trabalho.

Por exemplo: se o botão estiver copiado em dez arquivos diferentes e você quiser mudar sua aparência, terá de alterar dez pontos. Se existir um único componente `Botao`, a mudança acontece em um só lugar.

3.3 Organização

Componentes ajudam a separar melhor cada parte do projeto. Isso evita arquivos gigantescos e difíceis de entender.

3.4 Leitura mais fácil

Ler:

```
<Header />
<Menu />
<ListaProdutos />
<Rodape />
```

é muito mais claro do que encontrar tudo misturado em centenas de linhas dentro de um único arquivo.

3.5 Resumo em tabela

Vantagem	Como ajuda no projeto
Reutilização	Evita repetir o mesmo código em vários lugares.
Manutenção	Facilita corrigir, melhorar e atualizar partes da interface.
Organização	Divide o projeto em partes menores e mais compreensíveis.
Legibilidade	Torna o código mais claro e mais fácil de ler.
Escalabilidade	Ajuda o projeto a crescer sem virar bagunça.

4 Componente Funcional

4.1 O que é

O componente funcional é uma função JavaScript que retorna JSX.
Hoje em dia, esse é o tipo de componente mais usado em React moderno.

4.2 Exemplo básico

```
function Saudacao() {
  return <h1>Olá, turma!</h1>;
}
```

4.3 Por que ele é chamado de funcional?

Porque ele é escrito como uma função.
Ou seja:

- recebe informações, se necessário;
- executa lógica;
- retorna o que será renderizado na tela.

4.4 Exemplo com mais estrutura

```
function PerfilAluno() {
  return (
    <div>
      <h2>Ana</h2>
      <p>Aluno de React</p>
    </div>
  );
}
```

4.5 Características

O componente funcional:

- é simples de escrever;
- é o padrão mais comum atualmente;
- facilita organização e reutilização;
- pode ser criado com função tradicional ou arrow function.

5 Exportação de um Componente Funcional

Para usar um componente em outro arquivo, normalmente precisamos exportá-lo.
Existem diferentes formas de fazer isso.

5.1 Declaração de função e exportação no final

Essa é uma forma muito comum e bastante clara para iniciantes.

```
function Botao() {
  return <button>Enviar</button>;
}

export default Botao;
```

Como ler esse código

- Primeiro declaramos a função `Botao`;
- Depois exportamos esse componente no final com `export default Botao`.

5.2 Exportação direta na declaração

Também podemos exportar o componente logo no momento em que ele é criado:

```
export default function Botao() {
  return <button>Enviar</button>;
}
```

Como funciona

Aqui, a função já nasce exportada.

Vantagem

É uma sintaxe curta e bastante usada.

Observação importante

Esse formato funciona bem com função tradicional.

5.3 Declaração de constante e exportação no final

Também podemos criar o componente em uma constante:

```
const Botao = function() {
  return <button>Enviar</button>;
};

export default Botao;
```

Ou com arrow function:

```
const Botao = () => {
  return <button>Enviar</button>;
};

export default Botao;
```

Ideia

Primeiro declaramos a constante, depois exportamos no final.

Importante

Essa forma é muito usada com arrow function.

5.4 Por que na arrow function com constante não exportamos default diretamente?

```
export default const Botao = () => {
  return <button>Enviar</button>;
};
```

Esse código está **incorreto**.

Por quê?

Porque na sintaxe do JavaScript, não podemos usar `export default` diretamente antes da declaração de uma constante dessa forma.

Forma correta

```
const Botao = () => {
  return <button>Enviar</button>;
};

export default Botao;
```

Resumo

Com `function`, podemos fazer:

```
export default function Botao() { ... }
```

Mas com `const` + arrow function, o correto costuma ser:

```
const Botao = () => { ... };
export default Botao;
```

5.5 Exportação direta nomeada

Além do `default`, existe também a exportação nomeada.
Exemplo:

```
export function Botao() {
  return <button>Enviar</button>;
}
```

Também pode ser feita assim:

```
function Botao() {
  return <button>Enviar</button>;
}

export { Botao };
```

E com constante:

```
const Botao = () => {
  return <button>Enviar</button>;
};

export { Botao };
```

O que muda?

Na exportação nomeada, o componente é exportado pelo nome dele, e a importação precisa respeitar esse nome usando chaves.

6 Componente de Classe

6.1 O que é

Antes dos componentes funcionais dominarem o React moderno, os componentes de classe eram muito usados.

Eles são escritos usando a sintaxe de classe do JavaScript e normalmente herdam de `React.Component`.

6.2 Exemplo básico

```
import React, { Component } from 'react';

class Saudacao extends Component {
  render() {
    return <h1>Olá, turma!</h1>;
  }
}

export default Saudacao;
```

6.3 Como ler esse código

- Criamos uma classe chamada `Saudacao`;
- Essa classe herda de `Component`;
- Dentro dela existe o método `render()`;
- O que está dentro de `render()` é o que aparece na tela.

6.4 Observação importante

Hoje, em React moderno, os componentes funcionais são preferidos na maior parte dos casos. Mesmo assim, entender componentes de classe é importante porque:

- muitos projetos antigos ainda usam esse padrão;
- você pode encontrar esse formato em códigos já existentes;
- ajuda a compreender a evolução do React.

7 Exportação de um Componente de Classe

7.1 Declaração de classe e exportação no final

```
import React, { Component } from 'react';

class Perfil extends Component {
  render() {
    return (
      <div>
        <h2>Perfil do aluno</h2>
        <p>React com classe</p>
      </div>
    );
  }
}

export default Perfil;
```

Como funciona

Primeiro a classe é criada, depois exportada no final.

7.2 Exportação direta na declaração

Também podemos exportar a classe diretamente:

```
import React, { Component } from 'react';

export default class Perfil extends Component {
  render() {
    return (
      <div>
        <h2>Perfil do aluno</h2>
        <p>React com classe</p>
      </div>
    );
  }
}
```

Vantagem

A classe já nasce exportada, deixando o código mais curto.

7.3 Exportação direta nomeada

Também é possível exportar de forma nomeada:

```
import React, { Component } from 'react';

export class Perfil extends Component {
  render() {
    return (
      <div>
        <h2>Perfil do aluno</h2>
        <p>React com classe</p>
      </div>
    );
  }
}
```

Ou:

```
import React, { Component } from 'react';

class Perfil extends Component {
  render() {
    return (
      <div>
        <h2>Perfil do aluno</h2>
        <p>React com classe</p>
      </div>
    );
  }
}

export { Perfil };
```

Resumo

Assim como nos componentes funcionais, a classe também pode ser exportada como **default** ou **nomeada**.

8 Importação default x importação nomeada

Essa é uma das partes que mais confundem iniciantes.

8.1 Importação default

Quando um arquivo exporta algo com `export default`, podemos importar sem chaves.
Exemplo do arquivo `Botao.js`:

```
function Botao() {
  return <button>Enviar</button>;
}

export default Botao;
```

Importação:

```
import Botao from './Botao';
```

Importante

No `default`, o nome usado na importação pode até ser trocado:

```
import MeuBotao from './Botao';
```

Isso funciona, embora **não seja a melhor prática** se atrapalhar a clareza.

8.2 Importação nomeada

Quando usamos exportação nomeada, a importação precisa usar chaves.
Arquivo `Botao.js`:

```
export function Botao() {
  return <button>Enviar</button>;
}
```

Importação:

```
import { Botao } from './Botao';
```

Importante

Na importação nomeada:

- usamos chaves;
- o nome precisa ser o mesmo da exportação.

8.3 Comparação direta

Tipo	Exportação	Importação
Default	`export default Botao;`	`import Botao from ’./Botao’;`
Nomeada	`export { Botao };`	`import { Botao } from ’./Botao’;`

8.4 Exemplo com múltiplas exportações nomeadas

Arquivo:

```
export function Botao() {
  return <button>Enviar</button>;
}

export function Card() {
  return <div>Card</div>;
}
```

Importação:

```
import { Botao, Card } from './Componentes';
```

Vantagem

A exportação nomeada é útil quando um mesmo arquivo possui vários elementos exportados.

9 Esquema mental: default x nomeada

Exportação default

```
Arquivo exporta 1 elemento principal
↓
Importação sem chaves
↓
Uso mais comum para componente principal do arquivo
```

Exportação nomeada

```
Arquivo exporta 1 ou vários elementos pelo nome
↓
Importação com chaves
↓
Útil quando queremos exportar várias coisas no mesmo arquivo
```

10 Diferença entre componente funcional e componente de classe

Aspecto	Componente Funcional	Componente de Classe
Sintaxe	Função JavaScript	Classe JavaScript
Estrutura principal	Retorna JSX diretamente	Usa método `render()`
Leitura	Geralmente mais simples	Geralmente mais verbosa
Uso atual	Mais comum em React moderno	Mais comum em projetos antigos
Preferência atual	Alta	Menor

10.1 Resumo

Hoje, em projetos novos, o mais comum é usar **componentes funcionais**. Mas conhecer componentes de classe continua sendo importante para leitura de códigos antigos e para compreender a base histórica do React.

11 Componentes reutilizáveis e claros

11.1 Fazer uma única coisa bem feita

Uma boa prática é criar componentes pequenos e focados.

Por exemplo, em vez de criar um componente gigantesco que faz tudo, é melhor dividir em partes menores:

```
function Titulo() {
  return <h1>Loja Virtual</h1>;
}

function BotaoComprar() {
```

```
  return <button>Comprar</button>;
}

function CardProduto() {
  return (
    <div>
      <Titulo />
      <BotaoComprar />
    </div>
  );
}
```

11.2 Por que isso ajuda?

Porque componentes pequenos:

- são mais fáceis de entender;
- são mais fáceis de testar;
- são mais fáceis de corrigir;
- podem ser reutilizados em diferentes contextos.

11.3 Problema de componentes grandes demais

Quando um componente faz muitas coisas ao mesmo tempo:

- a leitura fica confusa;
- a manutenção fica difícil;
- aumentam as chances de erro;
- a reutilização fica ruim.

12 Nomenclatura consistente

12.1 O que é

Nomenclatura consistente significa usar nomes claros, intuitivos e padronizados.

12.2 Exemplos bons

- `BotaoEnviar`
- `CardProduto`
- `MenuLateral`
- `FormularioLogin`

12.3 Exemplos ruins

- `coisa`
- `teste1`
- `abc`
- `componenteNovo2`

12.4 Por que isso importa?

Quando os nomes são claros:

- o código se torna mais fácil de ler;
- outros desenvolvedores entendem mais rápido;
- fica mais fácil localizar responsabilidades.

12.5 Padrão importante em React

Componentes React normalmente usam **PascalCase**, ou seja, cada palavra começa com letra maiúscula:

```
function MeuComponente() {
  return <p>Exemplo</p>;
}
```

13 Documentação adequada

13.1 Por que documentar?

Nem sempre o código sozinho deixa tudo óbvio. Em projetos maiores, documentação ajuda outros desenvolvedores a entender:

- o propósito do componente;
- quando ele deve ser usado;
- o que ele recebe;
- o que ele renderiza.

13.2 Exemplo de comentário simples

```
/*
  Componente responsável por exibir
  um botão de envio em formulários.
*/
function BotaoEnviar() {
  return <button>Enviar</button>;
}
```

13.3 Documentação clara e concisa

Boa documentação:

- não precisa ser enorme;
- deve ser objetiva;
- deve explicar a função real do componente.

13.4 Exemplo ruim de documentação

```
/* componente */
```

Esse comentário praticamente não ajuda em nada.

13.5 Exemplo melhor

```
/*
  Exibe um card com nome e preço do produto.
  Deve ser usado na listagem principal da loja.
*/
function CardProduto() {
  return (
    <div>
      <h2>Notebook</h2>
      <p>R$ 3000</p>
    </div>
  );
}
```

14 Modularização bem feita

14.1 O que é modularização?

Modularizar significa dividir o sistema em partes menores e organizadas, chamadas módulos.
No React, isso geralmente significa:

- separar componentes em arquivos diferentes;
- evitar colocar tudo em um único arquivo;
- dividir responsabilidades.

14.2 Exemplo ruim: tudo em um arquivo

Imagine um arquivo com:

- menu;
- rodapé;
- formulário;
- lista de produtos;
- lógica de página;
- estilos misturados.

Esse arquivo rapidamente se torna difícil de ler e de manter.

14.3 Exemplo melhor: código modular

```
src/

 components/
    Header.js
    Footer.js
    Botao.js
    CardProduto.js

 pages/
    Home.js
```

```
App.js
index.js
```

14.4 Como isso arruma o código?

A modularização:

- separa responsabilidades;
- reduz arquivos gigantes;
- facilita encontrar cada parte;
- melhora a manutenção;
- ajuda no trabalho em equipe.

14.5 Exemplo prático

Arquivo `Botao.js`:

```
function Botao() {
  return <button>Comprar</button>;
}

export default Botao;
```

Arquivo `App.js`:

```
import Botao from './Botao';

function App() {
  return (
    <div>
      <h1>Loja</h1>
      <Botao />
    </div>
  );
}

export default App;
```

14.6 Resultado

Cada arquivo cuida de uma parte. O projeto fica mais limpo e o raciocínio fica mais fácil.

15 Boas práticas com componentes

1. Criar componentes pequenos e com responsabilidade clara.
2. Evitar arquivos gigantescos.
3. Usar nomes intuitivos.
4. Reaproveitar componentes sempre que fizer sentido.
5. Organizar componentes em pastas coerentes.
6. Documentar quando necessário.
7. Escolher um padrão de exportação e manter consistência.

16 Erros comuns:

1. **Confundir componente com arquivo qualquer.** Nem todo arquivo é um componente React.
2. **Esquecer de exportar o componente.** Se o componente não for exportado, ele não poderá ser importado em outro arquivo.
3. **Misturar importação default com nomeada.** `default` usa sem chaves; nomeada usa com chaves.
4. **Tentar usar** `export default const`**.** Essa forma está incorreta para declaração com constante.
5. **Criar componentes com nomes pouco claros.** Isso dificulta a leitura do projeto.
6. **Fazer um único componente enorme.** Isso atrapalha reutilização, manutenção e entendimento.
7. **Não usar PascalCase nos componentes.** Em React, o padrão é nomear componentes com letras maiúsculas no início de cada palavra.

17 Esquema geral do conteúdo

```
Componentes React

 Componente Funcional
    function
    const + function expression
    const + arrow function
    export default
    export nomeada

 Componente de Classe
    class extends Component
    método render()
    export default
    export nomeada

 Importações
    default -> sem chaves
    nomeada -> com chaves

 Boas práticas
     reutilização
     manutenção
     clareza
     documentação
     nomenclatura
     modularização
```

18 Conclusão

Aprender React de verdade não é apenas decorar sintaxe. O mais importante é compreender como organizar a aplicação em componentes claros, reutilizáveis e bem estruturados.

Esse é um passo muito importante para sair do nível de apenas copiar exemplos e começar a construir projetos React com mais autonomia, confiança e organização.

19 Exercícios

19.1 Conceitos

1. Explique com suas palavras o que é um componente React.
2. Diga por que componentes ajudam na organização do código.
3. Explique a diferença entre componente funcional e componente de classe.
4. Diga por que a modularização ajuda em projetos grandes.

19.2 Exportação e importação

1. Escreva um exemplo de componente funcional com `export default` no final.
2. Escreva um exemplo de componente funcional com exportação nomeada.
3. Explique a diferença entre importação default e importação nomeada.
4. Corrija o código abaixo:

```
export default const Botao = () => {
  return <button>Enviar</button>;
};
```

19.3 Leitura de código

Analise o código:

```
export function Card() {
  return (
    <div>
      <h2>Produto</h2>
      <p>Preço: R$ 100</p>
    </div>
  );
}
```

Responda:

1. Esse componente é funcional ou de classe?
2. Ele está usando exportação default ou nomeada?
3. Como ele deve ser importado em outro arquivo?

19.4 Boas práticas

1. Dê dois exemplos de nomes bons para componentes.
2. Explique por que comentários/documentação podem ser importantes.
3. Explique por que componentes pequenos costumam ser melhores que componentes gigantes.

20 Gabarito — Exercícios

20.1 Conceitos

1) O que é um componente React?

É uma parte reutilizável da interface escrita em React. Ele representa um pedaço da tela e ajuda a dividir a aplicação em partes menores.

2) Por que componentes ajudam na organização?

Porque eles separam a interface em blocos com responsabilidades específicas. Isso torna o código mais limpo, mais legível e mais fácil de manter.

3) Diferença entre componente funcional e componente de classe

O componente funcional é escrito como uma função JavaScript e retorna JSX. O componente de classe é escrito com a sintaxe de classe e usa o método `render()` para retornar JSX.

4) Por que a modularização ajuda?

Porque divide o projeto em arquivos e partes menores, facilitando leitura, manutenção, localização de código e trabalho em equipe.

20.2 Exportação e importação

1) Exemplo com export default no final

```
function Botao() {
  return <button>Enviar</button>;
}

export default Botao;
```

2) Exemplo com exportação nomeada

```
export function Botao() {
  return <button>Enviar</button>;
}
```

3) Diferença entre importação default e nomeada

- **Default**: importa sem chaves.
- **Nomeada**: importa com chaves.

Exemplos:

```
import Botao from './Botao';
```

```
import { Botao } from './Botao';
```

4) Correção do código

Código correto:

```
const Botao = () => {
  return <button>Enviar</button>;
};

export default Botao;
```

20.3 Leitura de código

1) Tipo de componente

É um componente funcional, pois foi criado com função.

2) Tipo de exportação

Está usando exportação nomeada, pois foi declarado com `export function Card()`.

3) Forma de importação

```
import { Card } from './Card';
```

20.4 Boas práticas

1) Exemplos de nomes bons

- `FormularioLogin`
- `CardProduto`

2) Por que documentação pode ser importante?

Porque ajuda outros desenvolvedores a entender rapidamente o propósito e o uso do componente, especialmente em projetos maiores.

3) Por que componentes pequenos costumam ser melhores?

Porque são mais fáceis de entender, corrigir, testar, reutilizar e manter.